Relatorio Final: Reconhecimento de Pokemon em Video

Disciplina: Machine Learning e Deep Learning - Engenharia de Software

Parte 5: Respostas e Analises dos Experimentos

1. Por que separar treino e validacao por video?
Eu testei treinar o modelo em dois cenarios distintos. No primeiro, a separacao foi feita de forma aleatoria por fotos (frames misturados). No segundo, a separacao ocorreu isolando videos inteiros.
Quando a divisao foi feita misturando os frames, a acuracia de validacao atingiu rapidamente valores proximos a 99%. No entanto, ao testar no video final (teste_final.mp4), o modelo apresentou muitas falhas. Quando separei por video inteiro, a acuracia de validacao foi menor (em torno de 93%), mas o modelo acertou muito mais no video de teste real.
Isso ocorre devido ao "vazamento de dados" (data leakage). Um video gravado a 30fps possui frames quase identicos (o frame 100 e o 101 sao visualmente a mesma coisa). Se o frame 100 cai no treino e o 101 na validacao, o modelo esta essencialmente "trapaceando", pois esta sendo validado em uma imagem que ele ja decorou no treino. A divisao por video previne isso e obriga o modelo a generalizar. O numero da acuracia gerado pelo split por video representa o desempenho real de forma muito mais correta.

2. O fine-tuning ajudou?
Avaliando o modelo apenas treinando a camada Dense final e, em seguida, aplicando o Fine-tuning (descongelando as ultimas 30 camadas da MobileNetV2), percebi uma diferenca notavel no grafico de validacao (curvas.png). Ao aplicar o Fine-tuning com a taxa baixa (1e-5), a Loss de validacao continuou caindo suavemente e a precisao geral melhorou, pois o modelo adaptou os filtros de bordas e texturas genericas da ImageNet para os formatos especificos dos Pokemons.
Porem, ao tentar o Fine-tuning com uma taxa alta (1e-3), o grafico mostrou um pico na Loss de validacao e a acuracia despencou. A taxa precisa ser extremamente baixa no Fine-tuning para evitar o "esquecimento catastrofico". Como os pesos da MobileNet ja vieram bem treinados, passos grandes na descida do gradiente (1e-3) destroem esses pesos pre-existentes, destruindo o aprendizado basico.

3. O modelo reconhece o Pokemon de qualquer angulo?
Retreinei o modelo removendo propositalmente todos os frames em que o Charmander aparece de costas. No teste do detectar.py com o video do Charmander de costas, o modelo falhou. O modelo previu "?" (devido ao meu limiar de confianca) ou confundiu o Charmander com a classe fundo.
Isso prova que a IA baseia sua predicao em caracteristicas espaciais visiveis (como os olhos, barriga amarela e boca). Sem ver a silhueta de costas durante o treino, ela nao tem referencia matematica para aquela pose.
Duas estrategias que proponho para tornar o modelo mais robusto a angulos novos sao:
- Data Augmentation agressivo: Aplicar rotacoes e espelhamentos horizontais intensos durante o treino para simular posicoes inexistentes.
- Enriquecer o dataset: Gravar os videos garantindo a filmagem em 360 graus sob diferentes condicoes de iluminacao (o que apliquei no meu script final).

Relatorio Final: Reconhecimento de Pokemon em Video

Disciplina: Machine Learning e Deep Learning - Engenharia de Software

4. O que acontece com um Pokemon que o modelo nao conhece?
Apresentei ao video de teste um Pokemon novo (ex: Squirtle).
Sem minhas melhorias, a funcao Softmax forcaria a predicao para um dos 3 Pokemons conhecidos (pois a soma das saidas deve ser sempre 100%). A rede diria, com 50% de certeza, que eh um Bulbasaur.
Mas com a minha melhoria no detectar.py (Limiar de Confianca em 0.60 e Suavizacao de Janela), as probabilidades flutuaram (ficaram fracas e divididas) e a media caiu para menos de 60%. O codigo entao exibe "?". A classe "fundo" ajuda a capturar pixels genericos e impede chutes erroneos caso nao haja um alvo definido.

5. Classificacao ou deteccao?
Quando coloquei dois Pokemons lado a lado no mesmo quadro do video, o script detectar.py comecou a "piscar" rapidamente entre os nomes dos dois ou focou naquele que ocupava a maior area da imagem.
Isso acontece porque eu apliquei uma Classificacao de Imagem, que analisa todos os pixels e fornece apenas um rotulo geral para a cena.
Para resolver isso, eu precisaria usar Deteccao de Objetos (como o modelo YOLOv8).
- Nos dados (anotacao): Eu precisaria desenhar bounding boxes ao redor de cada Pokemon e exportar as coordenadas (X e Y) correspondentes.
- No modelo: Teria que alterar a rede neural para prever multiplas regioes de interesse simultaneamente em vez de usar uma camada global densa, o que permitiria encontrar e enquadrar os dois Pokemons independentemente.

Linha do Tempo gerada pelo script de reconhecimento:
 0.0s a 19.0s -> bulbasaur
19.0s a 36.9s -> pikachu
37.2s a 38.0s -> charmander
38.3s a 39.2s -> bulbasaur
40.6s a 44.9s -> charmander
44.9s a 54.4s -> charmander
Pokemon encontrados no video: ['bulbasaur', 'charmander', 'pikachu']